Worked Examples – Neural Models: Learning, Depth, Activations, and Output Layers

August 7, 2026

These examples connect two complementary views of neural models. From the *AI science* perspective, we ask what representation and learning mechanisms are required for a model to exhibit useful behaviour. From the *AI engineering* perspective, we ask how gradients, activations, losses, and numerical procedures let us build and debug such systems in practice.

Worked Example 1: Derivative as local sensitivity and the direction of learning

Problem. Consider the scalar loss

$$L(\theta) = (\theta - 3)^2.$$

At $\theta = 5$:

(a) Interpret $\frac{dL}{d\theta}$ as a local sensitivity.

(b) Use a first-order approximation to predict the loss at $\theta = 5.01$.

(c) If gradient descent uses learning rate $\eta = 0.1$, compute one update and check whether the loss falls.

(d) Explain why backpropagation and gradient descent are not the same algorithm.

Solution. The derivative is

$$\frac{dL}{d\theta} = 2(\theta - 3),$$

so at $\theta = 5$,

$$\left.\frac{dL}{d\theta}\right|_{\theta=5} = 4.$$

Thus a small increase ϵ in θ changes the loss by approximately 4ϵ. For $\epsilon = 0.01$,

$$L(5.01) \approx L(5) + 4(0.01) = 4 + 0.04 = 4.04.$$

The exact value is

$$L(5.01) = (2.01)^2 = 4.0401,$$

so the first-order prediction is very accurate for this small perturbation.
Gradient descent steps *against* the gradient:

$$\theta_{\text{new}} = \theta - \eta\frac{dL}{d\theta} = 5 - 0.1(4) = 4.6.$$

The new loss is

$$L(4.6) = (4.6 - 3)^2 = 2.56 < 4.$$

Backpropagation supplies derivatives such as $\frac{\partial L}{\partial \theta_i}$. Gradient descent uses those derivatives to change parameters. In short,

backpropagation computes gradients; gradient descent updates parameters.

Discussion. The derivative is best read as an "exchange rate" between a small parameter change and the resulting loss change. A positive derivative means that locally increasing the parameter increases the loss, so a descent step moves the parameter in the negative direction. The first-order approximation is local: a learning rate that is too large can leave the region where the linear approximation is useful and may overshoot.

Worked Example 2: Multiply along paths, add across paths

Problem. A scalar s affects a loss through two routes:

$$p = 3s, \qquad q = s^2, \qquad L = p + q.$$

At $s = 2$:

(a) Compute L.

(b) Compute $\frac{dL}{ds}$ by treating the expression as a computational graph.

(c) Verify the answer by first simplifying L as a direct function of s.

(d) Explain why the "add across paths" rule matters for shared parameters in larger networks.

Solution. At $s = 2$,

$$p = 6, \qquad q = 4, \qquad L = 10.$$

The contribution through p is

$$\frac{\partial L}{\partial p}\frac{\partial p}{\partial s} = 1 \cdot 3 = 3,$$

while the contribution through q is

$$\frac{\partial L}{\partial q}\frac{\partial q}{\partial s} = 1 \cdot 2s = 4.$$

Since s reaches L through both routes, the contributions add:

$$\frac{dL}{ds} = 3 + 4 = 7.$$

Directly,

$$L = 3s + s^2 \quad \Rightarrow \quad \frac{dL}{ds} = 3 + 2s = 7$$

at $s = 2$, confirming the result.

Discussion. This example contains the two bookkeeping rules that scale to backpropagation on large graphs:

multiply sensitivities along one path, and add contributions across paths.

A shared weight can influence the loss at many places in a computation. Its final gradient is the sum of all those influences. The same principle handles repeated embeddings, recurrent weights used at multiple time steps, and residual/skip connections.

Worked Example 3: Reverse-mode automatic differentiation on a linear neuron

Problem. For one training example, let

$$z = wx, \qquad s = z + b, \qquad r = s - y, \qquad L = r^2.$$

Use

$$w = 2, \quad x = 3, \quad b = 1, \quad y = 5.$$

(a) Perform the forward pass.

(b) Starting from $\bar{L} = \frac{\partial L}{\partial L} = 1$, propagate reverse-mode sensitivities and obtain $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$.

(c) Explain why reverse mode is attractive when a model has many parameters but only one scalar loss.

Solution. The forward pass is

$$z = 2 \cdot 3 = 6, \qquad s = 6 + 1 = 7, \qquad r = 7 - 5 = 2, \qquad L = 2^2 = 4.$$

Now propagate backwards. Since $L = r^2$,

$$\bar{r} = \frac{\partial L}{\partial r} = 2r = 4.$$

Because $r = s - y$,

$$\bar{s} = \bar{r}\frac{\partial r}{\partial s} = 4 \cdot 1 = 4.$$

Because $s = z + b$, both incoming arguments receive sensitivity 4:

$$\bar{z} = 4, \qquad \bar{b} = \frac{\partial L}{\partial b} = 4.$$

Finally $z = wx$, so

$$\bar{w} = \bar{z}\frac{\partial z}{\partial w} = 4x = 12,$$

and therefore

$$\boxed{\frac{\partial L}{\partial w} = 12, \qquad \frac{\partial L}{\partial b} = 4.}$$

Quantity	Forward value	Reverse sensitivity
z	6	$\bar{z} = 4$
s	7	$\bar{s} = 4$
r	2	$\bar{r} = 4$
L	4	$\bar{L} = 1$

Discussion. Forward-mode AD seeded at one input gives derivatives of all downstream quantities with respect to that one input. If there are n parameters, obtaining a full scalar-loss gradient this way requires one direction per parameter (or carrying an n-dimensional derivative object). Reverse mode instead seeds the single scalar output and, after the forward evaluation, obtains sensitivities with respect to all parameters in one reverse sweep. The trade-off is memory: values from the forward pass must often be retained, or recomputed, so that local derivatives can be evaluated during the backward sweep.

Worked Example 4: Why depth needs nonlinearity: the XOR case

Problem. A network is formed by stacking affine layers,

$$\boldsymbol{h}^{(\ell)} = W^{(\ell)}\boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}, \qquad \boldsymbol{h}^{(0)} = \boldsymbol{x},$$

with no nonlinear activation between layers.

(a) Show for two layers that the whole network is still one affine map.

(b) Explain why adding more such affine layers cannot solve XOR with a single linear decision boundary.

(c) Show how two nonlinear hidden units can construct useful intermediate features for XOR using a step activation.

Solution. For two affine layers,

$$\begin{aligned}\boldsymbol{h}^{(1)} &= W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)},\\ \boldsymbol{h}^{(2)} &= W^{(2)}\boldsymbol{h}^{(1)} + \boldsymbol{b}^{(2)}.\end{aligned}$$

Substituting gives

$$\begin{aligned}\boldsymbol{h}^{(2)} &= W^{(2)}\big(W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\big) + \boldsymbol{b}^{(2)}\\ &= \underbrace{W^{(2)}W^{(1)}}_{W_{\text{eff}}}\boldsymbol{x} + \underbrace{\big(W^{(2)}\boldsymbol{b}^{(1)} + \boldsymbol{b}^{(2)}\big)}_{\boldsymbol{b}_{\text{eff}}}.\end{aligned}$$

Thus the composition is just another affine map. Repeating the argument shows that any depth of affine-only layers collapses to one effective affine transformation.
XOR is not linearly separable in the original two-dimensional input space, so an affine-only network cannot create a representation in which a single linear threshold separates the classes.
With a step function, define

$$h_1 = \text{step}(x_1 + x_2 - 0.5), \qquad h_2 = \text{step}(x_1 + x_2 - 1.5),$$

then

$$\hat{y} = \text{step}(h_1 - h_2 - 0.5).$$

The hidden features behave as follows:

x_1	x_2	h_1	h_2	$\hat{y}$	XOR
0	0	0	0	0	0
0	1	1	0	1	1
1	0	1	0	1	1
1	1	1	1	0	0

Here h_1 behaves like OR and h_2 behaves like AND. The output combines these nonlinear intermediate features to compute XOR.

Discussion. Depth by itself is not the source of expressive power here; the nonlinear transformation between affine layers is essential. Modern training usually uses differentiable or almost-everywhere differentiable activations such as sigmoid, tanh, ReLU, leaky ReLU, or GELU. Their choice affects both *representation* and *optimisation*. For example, sigmoid derivatives satisfy $\sigma'(t) \leq 0.25$, so repeated products can shrink gradients; an active ReLU contributes derivative 1, although an inactive ReLU contributes 0.

Worked Example 5: Softmax, cross-entropy, and the logit gradient

Problem. A hidden layer has already produced

$$\boldsymbol{h} = \begin{bmatrix} 0.6225 \\ 0.3430 \end{bmatrix}.$$

A three-class output layer uses

$$W^{(2)} = \begin{bmatrix} 0.3 & -0.5 \\ -0.2 & 0.4 \\ 0.1 & 0.2 \end{bmatrix}, \qquad \boldsymbol{b}^{(2)} = \begin{bmatrix} 0.2 \\ 0.0 \\ -0.1 \end{bmatrix}.$$

The logits and softmax probabilities are approximately

$$\boldsymbol{z} = (0.2152, 0.0127, 0.0308), \qquad \boldsymbol{p} = (0.3776, 0.3084, 0.3140).$$

The true class is class 1, so $\boldsymbol{y} = (1, 0, 0)$.

(a) Compute the cross-entropy loss.

(b) Use the softmax–cross-entropy simplification to compute $\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}}$.

(c) Interpret the sign of each component.

(d) Compute $\frac{\partial \mathcal{L}}{\partial W^{(2)}}$.

(e) Explain why adding the same constant to all three logits does not change the prediction probabilities.

Solution. For a one-hot target, cross-entropy is

$$\mathcal{L} = -\sum_{k=1}^{3} y_k \log p_k = -\log p_1.$$

Hence

$$\mathcal{L} = -\log(0.3776) \approx 0.9739.$$

For softmax followed by cross-entropy,

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}} = \boldsymbol{p} - \boldsymbol{y},$$

so

$$\boxed{\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}} = (-0.6224,\ 0.3084,\ 0.3140).}$$

The correct-class component is negative. A gradient-descent update therefore tends to *increase* z_1. The other components are positive, so gradient descent tends to decrease the incorrect-class logits.

Because $\boldsymbol{z} = W^{(2)}\boldsymbol{h} + \boldsymbol{b}^{(2)}$,

$$\frac{\partial \mathcal{L}}{\partial W^{(2)}} = (\boldsymbol{p} - \boldsymbol{y})\boldsymbol{h}^{T}.$$

Numerically,

$$\frac{\partial \mathcal{L}}{\partial W^{(2)}} \approx \begin{bmatrix} -0.3875 & -0.2135 \\ 0.1919 & 0.1058 \\ 0.1955 & 0.1077 \end{bmatrix}.$$

(Entries are rounded.)
Finally, for any constant c,

$$\frac{e^{z_k+c}}{\sum_j e^{z_j+c}} = \frac{e^c e^{z_k}}{e^c \sum_j e^{z_j}} = \frac{e^{z_k}}{\sum_j e^{z_j}},$$

so softmax is invariant to adding the same constant to every logit. This is why implementations may subtract $\max_j z_j$ before exponentiating to improve numerical stability.

Discussion. The output layer and loss should be chosen together to match the task: linear output with squared error for a real-valued target; sigmoid with binary cross-entropy for one yes/no target; softmax with cross-entropy for one of K classes. For a minibatch, the loss gradient is simply the average of the example-wise gradients. In large language models, next-token prediction is the same mathematical classification pattern with K equal to the vocabulary size and one prediction made at many sequence positions.

Take-away ideas

- Learning is optimisation of a loss; backpropagation efficiently supplies the required derivatives.
- Computational graphs make the chain rule local: multiply along a path, add across paths.
- Reverse-mode AD is well suited to scalar-loss models with very many parameters.
- Nonlinear activations make depth representationally meaningful, but their derivatives also shape trainability.
- The output activation and loss encode the prediction task and determine the final error signal propagated into the network.

Reference

The terminology is consistent with standard treatments of neural networks and learning in S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., together with standard deep-learning treatments of backpropagation and automatic differentiation.